PhysAgent: Physics-Informed SDevice Code Generation via Ontology-Guided LLM

Shinbin Kim*
Dept. of Semicond. Eng.
POSTECH
Pohang, Rep. of Korea
sinbin0701@postech.ac.kr

Giung Kim*
Dept. of Semicond. Eng.
POSTECH
Pohang, Rep. of Korea
giung0408@postech.ac.kr

Jiwon Lee†
Dept. of Semicond. Eng.
POSTECH
Pohang, Rep. of Korea
jiwonlee@postech.ac.kr

Dae-Hwan Kang†
Dept. of Semicond. Eng.
POSTECH
Pohang, Rep. of Korea
daehwankang@postech.ac.kr

***Abstract*—As semiconductor scaling reaches its limits, Technology Computer-Aided Design (TCAD) simulation becomes essential for Design-Technology Co-Optimization (DTCO). However, the core challenge lies in configuring SDevice—the physics-and-solver specification stage of Sentaurus TCAD—rather than in device structure generation. We propose PhysAgent, which builds a Sentaurus TCAD guide-based ontology and integrates literature-driven domain knowledge to generate physically grounded SDevice files without relying on synthetic training data.**

***Keywords*—TCAD, ontology, physics-informed modeling, parameter file generation, domain knowledge, semiconductor materials**

I. Introduction

Recent studies have applied LLMs to automate TCAD code generation, primarily targeting structure files written for SDE (Sentaurus Structure Editor). These approaches rely on supervised fine-tuning (SFT) with synthetic datasets constructed by varying numerical parameters in a small number of baseline scripts [3]–[6]. However, this limits generalization to unseen device architectures or physical configurations. More critically, none adequately addresses physics model selection or parameter file generation for SDevice—the core of physically meaningful TCAD simulation. Even AgenticTCAD [6], which generates SDevice code, uses only 2–3 fixed physics configurations and omits parameter files entirely.

We propose PhysAgent, which constructs a Sentaurus TCAD guide-based ontology encoding physical models, equations, and their interdependencies, complemented by literature-driven material-specific knowledge. By providing this structured context to general-purpose LLMs, PhysAgent generates physically grounded SDevice files with explicit reasoning about model selection and parameter settings—without domain-specific fine-tuning.

II. Related Work

TCAD simulation consists of SDE for device geometry and SDevice for physics models and solver settings. While LLMs have been applied to SDE code generation through fine-tuning or in-context learning, SDevice configuration—selecting and parameterizing models such as mobility, recombination, and bandgap narrowing—remains unaddressed due to its high expertise requirements.

*These authors contributed equally to this work.
†Corresponding authors.

III. Proposed Framework

PhysAgent combines a TCAD physics ontology and literature-driven domain knowledge to guide LLM-based SDevice file generation. The constructed ontology is publicly available as an interactive web platform [7].

A. TCAD Physics Ontology

An SDevice simulation takes a command file (`.cmd`) and a parameter file (`.par`) as input. The physics block in the `.cmd` file specifies the physical models activated for each material, while the `.par` file defines the parameters required by these models along with fundamental properties such as bandgap, permittivity, and the DOS effective masses of electrons and holes. Thus, the model selection in the `.cmd` file determines the parameter set that must be provided in the `.par` file. To consistently capture this `.cmd`–`.par` dependency, we structure the relevant parameters, TCAD syntax, and physical knowledge into an ontology grounded in the governing equations of the physical models and the Poisson and transport equations. Each parameter is represented as a node, and nodes associated with the same physical model are grouped into an equation group. Node–equation and equation–equation relationships are encoded as edges and relations, from which the required parameter set is derived upon model selection. A syntax layer then applies the SDevice syntax to the selected equations and nodes to generate the `.cmd` and `.par` files (Fig. 1).

B. Ontology-Guided Generation

Given a target device and material, PhysAgent accesses the ontology through an MCP (Model Context Protocol) interface that exposes four tools. `ontology_overview` returns the node/edge map of the graph, serving as the entry point for grasping the overall structure. `get_chapter` identifies the equation-level relations associated with a selected node and derives the parameter set required by the corresponding physical equations. `get_reference` returns values and source citations for the derived parameters, drawn from a curated collection of papers, material datasheets, and TCAD references. Finally, `build_syntax` assembles the selected equations and parameter values under the SDevice syntax to generate the final `.cmd` (Physics block) and `.par` files. In our experiments, we employ Claude Sonnet, whose context

window is limited to 200k tokens, whereas the full ontology measures ∼458k tokens and therefore cannot be injected directly into the prompt. The MCP-based selective retrieval scheme circumvents this limitation by allowing the LLM to query only the information required at each step, keeping the per-query context well within the 200k-token budget. Moreover, each parameter in the generated `.par` inherits the citation returned by `get_reference`, thereby enabling parameter-level provenance (Fig. 2).

IV. Experimental Validations of PhysAgent

We compare three knowledge-source conditions under identical model, prompt, and task settings, each executed in 10 independent trials on a Si nMOSFET (`.cmd` Physics block + `.par` generation): (i) no external grounding (`no_source`), (ii) a raw-text dump of Sentaurus manual excerpts (SDevice manual), and (iii) the ontology accessed through the four tools of §III.B (`ontology_mcp`, the PhysAgent configuration). Decks are evaluated by (a) `sdevice` parser acceptance (structural validity; convergence and IV match are out of scope) and (b) coverage of 14 canonical `.par` sections (e.g., Bandgap, Epsilon, ConstantMobility). `no_source` hallucinates nonexistent keywords (0/10 pass, 39% coverage); SDevice manual occupies ∼88% of the 200K context window and improves coverage to 54% but still fails every parser check (0/10), as the LLM cannot reliably lift structural keywords from prose (Fig. 3). Only `ontology_mcp` achieves both parser pass and highest coverage (10/10, 81%), through per-model activation syntax (`get_chapter`), equation-anchored literature values (`get_reference`, Fig. 4), and an explicit section checklist (`ontology_overview`). The **structure—not the volume—of context** sets the quality ceiling in LLM-driven TCAD deck generation.

V. Conclusion

We presented PhysAgent, which generates physically grounded SDevice files by combining a TCAD guide-based ontology with literature-driven domain knowledge, enabling general-purpose LLMs to support expert-level TCAD modeling without synthetic training data. On a Si nMOSFET benchmark, PhysAgent achieves 10/10 parser pass at 81% section coverage, versus 0/10 parser pass for both raw-text and no-context baselines.

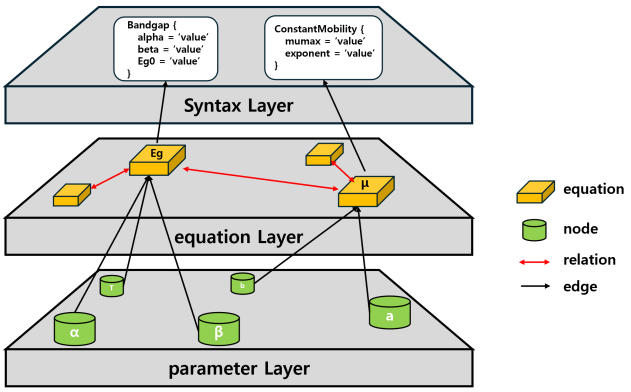

Fig. 1. TCAD physics ontology (678 equations, 1,631 parameters, 216 physical models, 30 literature references): node and edge types capturing inter-model dependencies.

Fig. 2. PhysAgent framework: LLM agents access the ontology graph via MCP tool calls to generate physically grounded parameter files.

Mode	t1	t2	t3	t4	t5	t6	t7	t8	t9	t10	Dominant failure
`no_source`	K	K	K	K	K	K	K	K	K	K	Hallucinated keyword
`manual`	V	M	V	V	V	M	M	M	V	M	Malformed syntax
PhysAgent (`ontology_mcp`)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	All pass

Fig. 3. Successful TCAD execution: raw trial-by-trial outcomes (t1–t10, not aggregated) across knowledge-source modes on a Si nMOSFET. K: hallucinated keyword; V: invalid value; M: malformed syntax/section.

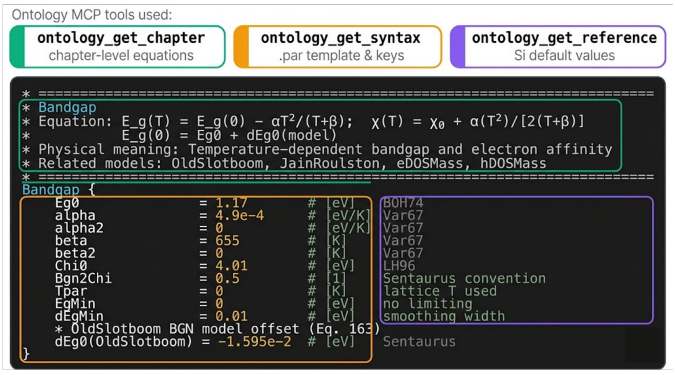

Fig. 4. PhysAgent workflow for ontology-grounded `.par` file generation: chapter-level equations, parameter templates, and material-specific reference values are assembled through `ontology_get_chapter`, `ontology_get_syntax`, and `ontology_get_reference`, illustrated for the Bandgap and HighFieldDependence blocks.

References

[1] Z. Ji *et al.*, "Toward reliability- and variability-aware design-technology co-optimization in advanced nodes," *IEEE Trans. Electron Devices*, vol. 71, no. 1, pp. 138–150, Jan. 2024.

[2] Synopsys, "Sentaurus TCAD," 2025. [Online]. Available: https://www.synopsys.com/manufacturing/tcad.html

[3] L. M. L. Nguyen, A. Lu, and H. Y. Wong, "TCAD structure input file generation using large language model," in *Proc. SISPAD*, 2024, pp. 1–4.

[4] Z. Zhang, S. Nassif, and M. Tahoori, "ChatTCAD: Leveraging large language models for automated TCAD simulation file generation," in *Proc. ISVLSI*, 2025, pp. 1–6.

[5] J. Dou *et al.*, "MALTS: A multi-agent large language model-based layout-to-TCAD structure synthesizer," in *Proc. ISEDA*, 2025, pp. 795–800.

[6] G. Fan *et al.*, "AgenticTCAD: A LLM-based multi-agent framework for automated TCAD code generation and device optimization," *arXiv:2512.23742*, 2026.

[7] S. Kim and G. Kim, "TCAD Physics Ontology," 2026. [Online]. Available: https://ontology-tcad.vercel.app/

[8] W. Bludau, A. Onton, and W. Heinke, "Temperature dependence of the band gap of silicon," *J. Appl. Phys.*, vol. 45, no. 4, pp. 1846–1848, Apr. 1974.

[9] Y. P. Varshni, "Temperature dependence of the energy gap in semiconductors," *Physica*, vol. 34, no. 1, pp. 149–154, 1967.